

Refactorización

Principios, patrones y limitaciones

Idea clave: ¿qué es refactorizar?

Refactorizar es mejorar el diseño interno del código — cómo está escrito y organizado — sin cambiar lo que hace: misma funcionalidad, mismos resultados.

El programa sigue haciendo exactamente lo mismo

✓ Qué mejora

- El código es más claro y legible
- Es más fácil de mantener y ampliar
- Es más fácil de probar

= Qué NO cambia

- El resultado del programa
- La funcionalidad para el usuario
- Los datos de entrada y salida

⚠ Si el comportamiento cambia, ya no es un refactor — es un cambio de funcionalidad.

Refactorización vs optimización

	Optimización	Refactorización
Qué mejora	Rendimiento o uso de recursos	Estructura y legibilidad del código
¿Cambia el resultado?	No	No
¿Cuándo se hace?	Cuando hay un problema medido de rendimiento	Cuando el código es difícil de leer o mantener
Ejemplo	Sustituir List por HashSet para búsquedas rápidas	Renombrar f() por calcularPrecioConIVA()

Orden habitual en proyectos reales: primero se refactoriza para que el código sea legible, y solo entonces se optimiza donde haga falta.

Principios básicos de refactorización

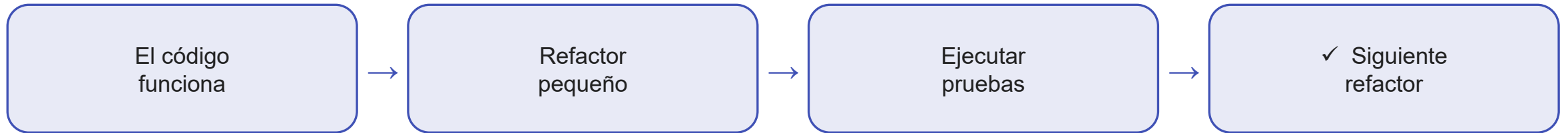
Estos cuatro principios están detrás de casi todas las refactorizaciones. No hace falta aplicarlos todos a la vez: basta con identificar el más evidente y trabajar en pasos pequeños.

Principio	En qué consiste
Código legible	Nombres que describen la intención. Si tienes que comentar qué hace una variable, es señal de que el nombre no es bueno.
Responsabilidad única	Cada método o clase hace una sola cosa. Un método que valida, calcula y envía emails es difícil de entender y de probar.
Evitar duplicación (DRY)	Si copias y pegas lógica, cualquier bug aparece en varios sitios a la vez. Una regla → un único lugar en el código.
Cambios pequeños y verificados	Refactorizar en pasos pequeños y comprobar que todo sigue funcionando. Un refactor enorme es difícil de revertir si algo falla.

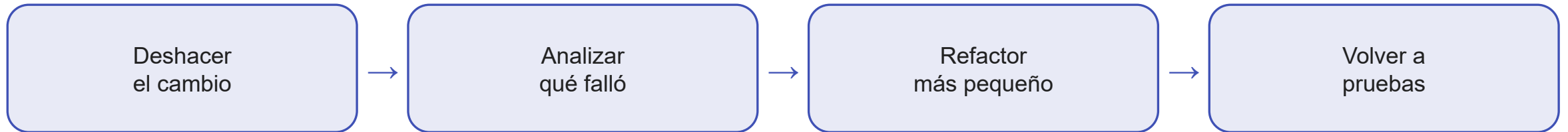
Regla de oro: refactorizar con pruebas

Refactorizar implica tocar código que funciona, y eso siempre tiene riesgo. La manera segura es tener verificaciones — pruebas automatizadas o casos manuales — para detectar si algo ha dejado de funcionar.

Camino normal (pruebas pasan):



Si las pruebas fallan:



⚠ Sin pruebas: trabaja en pasos aún más pequeños y verifica manualmente después de cada cambio. Lo ideal es añadir pruebas básicas antes de refactorizar zonas críticas.

Patrones: Rename y Extract Constant

1) Rename — Renombrar

Nombres crípticos que no explican la intención. En dos semanas nadie sabe qué significa d o f.

✗ Antes

```
int d = 0;
double f(double p) {
    return p * 1.21;
}
```

✓ Después

```
int totalPedidos = 0;
double calcularPrecioConIVA(
    double precio) {
    return precio * 1.21;
}
```

□ **Atajo: Shift + F6 — actualiza todas las referencias**

3) Extract Constant

Números mágicos sueltos sin explicación. Si la regla cambia, hay que buscarlos en varios sitios y siempre se olvida alguno.

✗ Antes

```
if (password.length() < 8)
    throw new IllegalArgumentException();

// más abajo, el mismo 8...
if (newPass.length() < 8)
    errores.add("Contraseña corta");
```

✓ Después

```
private static final int
    MIN_PASSWORD_LENGTH = 8;

if (password.length() < MIN_PASSWORD_LENGTH)
    throw new IllegalArgumentException();
if (newPass.length() < MIN_PASSWORD_LENGTH)
    errores.add("Contraseña corta");
```

Si cambia el mínimo a 10, se cambia en un único sitio.

Patrón: Extract Method — Extraer método

Un método largo obliga a leer todo para entender una parte. Si tiene más de 10-15 líneas, suele estar haciendo demasiadas cosas a la vez.

✗ Antes — todo mezclado

```
void registrarUsuario(String email,
    String pass) {
    // validar email
    if (email == null || !email.contains("@"))
        throw new IllegalArgumentException();
    // validar contraseña
    if (pass == null || pass.length() < 8)
        throw new IllegalArgumentException();
    // guardar en BD
    bd.insert("usuarios", email, pass);
    // enviar correo
    correo.enviar(email, "Bienvenido", "...");
}
```

✓ Después — un método, una responsabilidad

```
void registrarUsuario(String email,
    String pass) {
    validarEmail(email);    // ← intención clara
    validarPassword(pass);
    guardarUsuario(email, pass);
    enviarBienvenida(email);
}
void validarEmail(String email) { ... }
void validarPassword(String pass) { ... }
void guardarUsuario(String e, String p) { ... }
void enviarBienvenida(String email) { ... }
```

Atajo IntelliJ: selecciona el bloque → Ctrl + Alt + M (Windows) / Cmd + Alt + M (Mac). El IDE crea el método y reemplaza el bloque automáticamente.

Patrón: Replace Conditional — Simplificar condición compleja

Una condición con cuatro o cinco partes unidas con && y || es difícil de leer de un vistazo y muy fácil de romper al modificarla.

✖ Antes

```
if (user != null
    && user.isActive()
    && user.getRole() != null
    && user.getRole().equals("ADMIN")
    && !user.isSuspended()) {
    mostrarPanelAdmin();
}
// Para entender qué se comprueba
// hay que leer las 5 líneas enteras
```

✔ Después

```
if (esAdminActivo(user)) {
    mostrarPanelAdmin();
}

boolean esAdminActivo(User user) {
    return user != null
        && user.isActive()
        && !user.isSuspended()
        && "ADMIN".equals(user.getRole());
}
```

Antes	Después
Sin nombre → hay que leerla entera para entenderla	esAdminActivo() dice exactamente qué se comprueba
No se puede probar la condición por separado	La función se puede cubrir con un test unitario

Patrón: DRY — Eliminar código duplicado

Copiar y pegar lógica parece rápido, pero cuando hay que corregir un error o cambiar una regla, hay que hacerlo en todos los sitios — y siempre se olvida alguno.

✗ Antes — misma lógica en 3 controladores

```
// RegistroController.java
if (email == null || !email.contains("@"))
    throw new IllegalArgumentException();

// ActualizarPerfilController.java
if (email == null || !email.contains("@"))
    throw new IllegalArgumentException();

// RecuperarPasswordController.java
if (email == null || !email.contains("@"))
    errores.add("Email no válido"); // ← mensaje
distinto
```

✓ Después — una clase de utilidad

```
// EmailValidator.java
public class EmailValidator {
    public static void validar(String email) {
        if (email == null || !email.contains("@"))
            throw new IllegalArgumentException(
                "Email no válido");
    }
}

// En cualquier controlador:
EmailValidator.validar(email);
```

DRY = Don't Repeat Yourself. Si mañana cambia la regla de validación, se cambia en un único sitio y se aplica en todos los controladores automáticamente.

Patrón: Move Method — Mover la lógica donde corresponde

El controlador web acaba haciendo de todo — validar, calcular, acceder a la BD — y se convierte en un cajón de sastre imposible de mantener.



✗ Antes — todo en el Controller

```
@PostMapping("/pedido")
public String crearPedido(
    String email, List<Double> precios) {
    if (email == null || !email.contains("@"))
        return "error";
    double total = 0;
    for (Double p : precios) total += p;
    total *= 1.21;           // IVA hardcodeado
    bd.save(new Pedido(email, total));
    return "ok: " + total;
}
```

✓ Después — responsabilidades separadas

```
// Controller — solo recibe y responde
@PostMapping("/pedido")
public String crearPedido(
    String email, List<Double> precios) {
    return pedidoService.crearPedido(email, precios);
}

// Service — lógica de negocio
public String crearPedido(String email,
    List<Double> precios) {
    validarEmail(email);
    double total = calcularTotalConIVA(precios);
    pedidoRepository.save(new Pedido(email, total));
    return "ok: " + total;
}
```

Limitaciones y riesgos al refactorizar

Refactorizar no es gratis. En proyectos reales hay que valorar si el esfuerzo merece la pena: un código que funciona bien — aunque no sea perfecto — puede no necesitar refactorización urgente.

✓ Buen momento para refactorizar	✗ Mal momento
Antes de añadir una nueva funcionalidad	Justo antes de una entrega con fecha fija
Cuando los tests pasan y hay tiempo	Sin ningún tipo de prueba que te proteja
En pasos pequeños con margen para verificar	Mezclado con un cambio de funcionalidad
El código va a crecer y a ser mantenido	Código que nadie va a volver a tocar

Riesgos frecuentes: Cambiar la firma de un método sin actualizar todos los usos · Refactorizar sin pruebas y romper casos límite · Mezclar refactor con cambio de funcionalidad: si algo falla, ya no sabes qué lo ha causado · Hacer un refactor enorme de golpe sin poder revertir pasos.

Herramientas del IDE para refactorizar

Los IDEs modernos incluyen refactorizaciones automáticas: en lugar de cambiar el nombre de un método a mano en todos los archivos, el IDE lo hace en un paso y actualiza todas las referencias, eliminando una fuente habitual de errores.

Acción	Qué hace	Windows / Linux	Mac
Rename	Renombra y actualiza todas las referencias	Shift + F6	Shift + F6
Extract Method	Convierte un bloque seleccionado en un método nuevo	Ctrl + Alt + M	Cmd + Alt + M
Extract Variable	Da nombre a una expresión inline	Ctrl + Alt + V	Cmd + Alt + V
Extract Constant	Como Variable, pero como constante de clase	Ctrl + Alt + C	Cmd + Alt + C
Inline	Operación inversa: integra de vuelta en el sitio de uso	Ctrl + Alt + N	Cmd + Alt + N
Move	Mueve una clase o método a otro paquete o clase	F6	F6

Usa siempre las herramientas del IDE: un buscar-y-reemplazar manual no distingue entre el nombre de un método y la misma cadena dentro de un comentario o un String.